

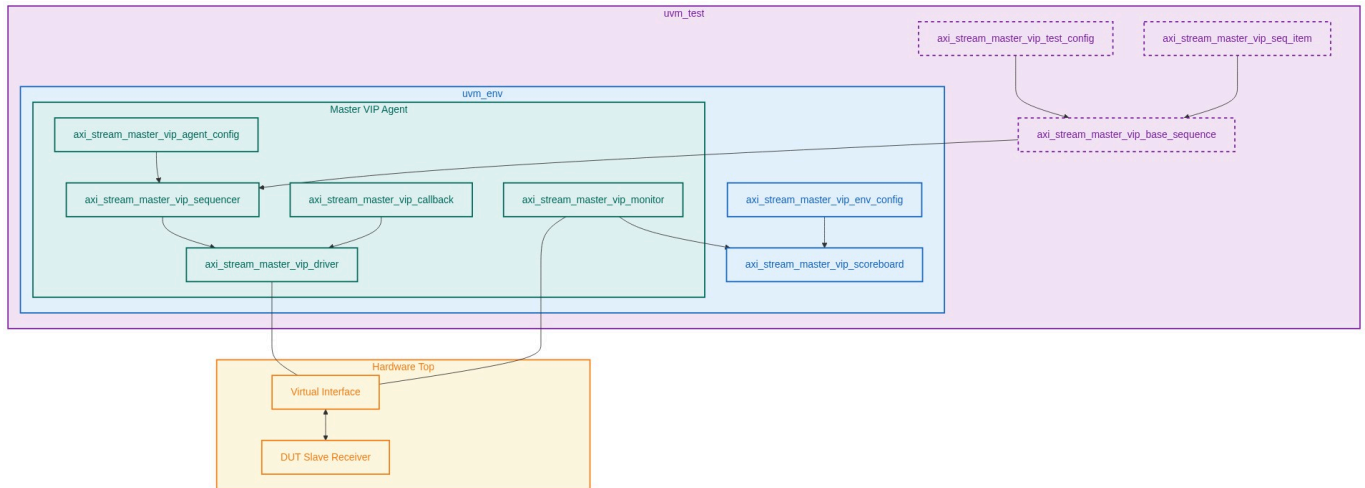
Verification Plan Report

Project: AXI-Stream Master VIP Verification Plan

Version: 1.0

Verification Role: Master

1. UVM Environment Architecture



2. Test Requirements

Index	Feature/Sub	Name	Description
REQ_MST_0 1	Handshake (TVALID Stability Post-Assertion)	TVALID Permanence Until Handshake	Hardware Logic: The AXI-Stream specification mandates that once a Transmitter asserts TVALID HIGH, it must remain asserted without interruption until the handshake completes ? i.e., until TREADY is sampled HIGH on a rising ACLK edge simultaneously with TVALID. Premature deassertion of TVALID constitutes a protocol violation and creates a race condition at the Receiver's data capture logic, potentially causing data corruption or missed transfers. All payload signals (TDATA, TSTRB, TKEEP, TLAST, TID, TDEST, TUSER) must remain stable and unchanged for the entire TVALID-asserted window. Verification Intent: The Master VIP driver must assert TVALID and hold all associated data signals stable until TREADY is observed HIGH. Directed test cases must verify the locked-data window under zero-stall, moderate-stall (5-20 cycles), and maximum-stall (100+ cycles) TREADY delay scenarios. The VIP monitor must implement a protocol checker that flags any mid-transfer TVALID deassertion or data signal instability as a UVM_ERROR.

Verification Plan Report

REQ_MST_0 2	Handshake (Pre-TVALID TREADY Assertion)	TREADY-Before-TVALID Handshake Completion	Hardware Logic: The AXI-Stream specification permits a Receiver to assert TREADY before the Transmitter drives TVALID. In this scenario, the DUT Slave Receiver pre-qualifies itself as ready to accept data, and the transfer completes the instant the Master VIP drives TVALID HIGH. This represents a single-cycle latency path and must be correctly handled by the Master VIP's driver state machine to avoid inserting unnecessary wait cycles after TREADY is already asserted. Verification Intent: Configure the DUT stimulus model to pre-assert TREADY. Verify that the Master VIP correctly drives TVALID and observes that the transfer completes on the very first cycle. Test with both single-beat and multi-beat packet scenarios to confirm the driver correctly advances to the next sequence item without inserting spurious idle cycles. Verify the scoreboard captures the exact beat count and payload.
REQ_MST_0 3	Handshake (Simultaneous TVALID and TREADY)	Zero-Stall Single-Cycle Transfer	Hardware Logic: When both TVALID and TREADY are asserted in the same ACLK cycle, the transfer completes immediately in that single cycle with zero wait states. This is the maximum throughput scenario and represents the critical path for back-to-back packet streams. Any additional latency introduced by the Master VIP driver in this scenario would constitute a throughput degradation defect. Verification Intent: Generate constrained-random sequences where the DUT stimulus model drives TREADY HIGH in the same cycle TVALID is asserted. Verify that the Master VIP achieves 100% channel utilization (no idle cycles between successive transfers) across a 256-beat burst. The scoreboard must verify byte-accurate data integrity and correct TLAST positioning under maximum throughput conditions. Measure throughput in transfers-per-cycle and assert it equals 1.0.

Verification Plan Report

REQ_MST_0 4	Packet Boundaries (TLAST Framing)	Packet Boundary Demarcation via TLAST	Hardware Logic: The TLAST signal is the sole explicit packet boundary marker in the AXI-Stream protocol. When TLAST is deasserted, it indicates another transfer can follow within the same packet. When asserted, it signals the final beat of the logical packet. The number of TLAST assertions between Transmitter and Receiver must be perfectly preserved ? any additional or missing assertion constitutes a framing error. The start of each new packet is implicitly defined as the first transfer after the preceding packet's TLAST (or the first transfer after reset for the initial packet). Verification Intent: Generate packets of varying lengths: single-beat (TLAST on beat 1), short-burst (2-8 beats), standard-burst (16-128 beats), and maximum-length burst (256 beats). The scoreboard must independently reconstruct packet boundaries from the TLAST signal and verify the packet byte count matches the expected sequence item. Verify back-to-back packets with no inter-packet idle cycles.
REQ_MST_0 5	Packet Boundaries (Null-Termination)	Zero-Data Packet Termination	Hardware Logic: The AXI-Stream specification explicitly permits a transfer to have TLAST asserted but contain zero data or position bytes (all TKEEP bits LOW). This null termination pattern is used to push data through intermediate FIFOs, indicate end-of-packet when no residual data exists, or complete an operation at an endpoint expecting a TLAST pulse. This is a legal but non-trivial scenario that exercises the DUT's buffer management and packet boundary detection logic. Verification Intent: Generate null-termination beats (TLAST=1, all TKEEP=0) as the final beat of multi-beat packets. Verify the DUT receiver acknowledges the null beat without consuming data. Verify the scoreboard correctly identifies the packet boundary based on TLAST and does not penalize the missing payload. Additionally verify merging rules: a null-TLAST beat with matching TID/TDEST can be merged with an earlier beat if that earlier beat has TLAST=0.

Verification Plan Report

REQ_MST_0 6	Data Packing (TKEEP Null-Byte Qualification)	Null Byte Stream Generation and Transport	Hardware Logic: TKEEP is the primary byte-validity qualifier. When TKEEP[x] is HIGH, the associated TDATA[(8x+7):8x] byte is a valid payload byte that must be transmitted to the destination. When TKEEP[x] is LOW, the associated byte is a null byte that can be stripped from the stream by any intermediate interconnect. A transfer where all TKEEP bits are LOW is legal and can be suppressed if TLAST is also deasserted. The Master VIP must generate all possible TKEEP configurations to fully exercise the DUT's null-byte handling logic. Verification Intent: Generate transfers with all TKEEP combinations: all-HIGH (dense stream), alternating-HIGH (50% utilization), sparse random patterns, and all-LOW (complete null transfer). Verify the DUT correctly extracts only the valid bytes. The scoreboard must track the expected data payload excluding all null positions and compare against what the DUT receives.
REQ_MST_0 7	Data Packing (TSTRB Position-Byte Qualification)	Position-Byte vs Data-Byte Discrimination	Hardware Logic: TSTRB provides a second-level qualification layer when TKEEP is HIGH. A byte where TKEEP=HIGH and TSTRB=HIGH is a data byte containing valid information. A byte where TKEEP=HIGH and TSTRB=LOW is a position byte that indicates the relative position of data bytes in the stream but carries no valid data content. The reserved and forbidden combination is TSTRB=HIGH with TKEEP=LOW; this must never be generated by a compliant Transmitter. Any interconnect may bypass the TDATA associated with a position byte since its content is not valid. Verification Intent: Generate four distinct scenarios per the Table 2-3 truth table: (1) data bytes (TKEEP=1, TSTRB=1), (2) position bytes (TKEEP=1, TSTRB=0), (3) null bytes (TKEEP=0, TSTRB=0). The negative scenario ? TSTRB=HIGH with TKEEP=LOW ? must be injected via a violation knob and the VIP monitor must flag a UVM_ERROR. Verify the scoreboard correctly differentiates between position bytes and data bytes for payload comparison.

Verification Plan Report

REQ_MST_0 8	Data Routing (TID Stream Interleaving)	Multi-Stream TID Interleaving and Context Isolation	Hardware Logic: The TID signal identifies the data stream of each transfer. Transfers within the same logical packet must share the same TID and TDEST values. However, the AXI-Stream specification permits interleaving of transfers belonging to different streams (different TID values) on a per-transfer basis, not just at TLAST boundaries. This allows the Master VIP to multiplex multiple logical streams over a single physical AXI-Stream channel, with each stream maintaining its own independent packet boundary tracking via TLAST assertions. Merging of transfers from different packets with different TID/TDEST values is explicitly forbidden. Verification Intent: Generate interleaved traffic across 2, 4, and 8 simultaneous TID streams. Verify that the scoreboard correctly demultiplexes packets by TID and reconstructs the byte stream for each. Verify that TLAST assertions are independently tracked per (TID, TDEST) pair. Inject a TID-change-within-packet violation (TID changes while TLAST=0) to verify the monitor reports a protocol error.
REQ_MST_0 9	Data Routing (Continuous Packet Mode)	Continuous Packet Protocol Constraints	Hardware Logic: When the Continuous_Packets property is True, the AXI-Stream interface operates under a stricter subset of rules. Transfer interleaving for different packets must not occur ? when TLAST is LOW, TID and TDEST must not change in the next transfer. Position bytes (TSTRB=LOW with TKEEP=HIGH) are not supported in this mode; TSTRB is absent. All TKEEP bits must be HIGH when TLAST is LOW (no null bytes within a continuous packet). Null bytes are only permitted above the data bytes on the final beat (TLAST=HIGH). This mode is supported on both AXI4-Stream and AXI5-Stream interfaces. Verification Intent: Enable Continuous_Packets mode via a configuration knob in agent_config. Verify that TID and TDEST remain stable across all intra-packet transfers. Verify that TKEEP is all-HIGH for every non-final beat. Inject a violation where TID changes mid-packet (TLAST=LOW) and verify the monitor flags a UVM_ERROR. Verify that TSTRB is not driven (or defaults HIGH) in this mode.

Verification Plan Report

REQ_MST_1 0	Reset (Reset Behavior and Recovery)	ARESETn Protocol Compliance and Post-Reset Recovery	Hardware Logic: The ARESETn signal is active-LOW and can be asserted asynchronously at any point. However, deassertion must be synchronous ? synchronized to the rising edge of ACLK. During the asserted reset condition, the Master VIP (Transmitter) must drive TVALID LOW. All other signals may take any value during reset. After ARESETn deassertion, the Transmitter must only begin driving TVALID HIGH at a rising ACLK edge that follows a rising edge at which ARESETn was sampled HIGH. This prevents spurious transfers that could corrupt post-reset state machines in the DUT. Verification Intent: Assert ARESETn LOW during idle, mid-transfer, and mid-packet scenarios. Verify TVALID is driven LOW in each case. Verify the Master VIP does not drive TVALID HIGH on the same cycle ARESETn goes HIGH ? it must wait at least one full ACLK cycle. The scoreboard must flush pending data when reset is detected and not report false comparison failures for in-flight data that was cancelled by reset. Verify multiple consecutive reset cycles with correct recovery behavior each time.
REQ_MST_1 1	Wake-Up Signaling (TWAKEUP Timing and Deadlock Prevention)	TWAKEUP Activity Indicator Compliance (AXI5)	Hardware Logic: The TWAKEUP signal, present on AXI5-Stream interfaces with the Wakeup_Signal property set True, indicates any activity associated with the AXI-Stream interface. It is intended to enable a clock controller to gate clocks and power for downstream components. The specification defines strict temporal rules: if TWAKEUP and TVALID are both HIGH in the same cycle, TWAKEUP must remain asserted until TREADY is asserted. A Receiver is permitted to wait for TWAKEUP before asserting TREADY ? creating a potential deadlock if TWAKEUP is never asserted. TWAKEUP must be glitch-free (derived from a register or glitch-free OR tree) and must be suitable for asynchronous sampling in a different clock domain. Verification Intent: Test TWAKEUP assertion at least one cycle before TVALID (recommended path). Test simultaneous TWAKEUP+TVALID assertion and verify TWAKEUP stays HIGH until TREADY arrives. Test TWAKEUP deasserted after all transfers complete. Inject a TWAKEUP assertion followed by no transfer (permitted but not recommended) and verify no deadlock occurs. Verify TWAKEUP is glitch-free across 1000 cycles by checking no mid-cycle transitions in the monitor.

Verification Plan Report

REQ_MST_1 2	Parity Protection (AXI5 Odd-Parity Check Signals)	Odd Parity Generation and Error Detection (AXI5)	Hardware Logic: AXI5-Stream interfaces with Check_Type set to Odd_Parity_Byte_All require the Transmitter to generate odd parity check signals for every driven output. The Master VIP must generate: TVALIDCHK (1-bit parity for TVALID), TDATACHK (TDATA_WIDTH/8 bits, one parity bit per byte), TSTRBCHK, TKEEPCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, and TWAKEUPCHK. Check signals must be driven correctly in every cycle where their Check Enable condition is TRUE (ARESETn for TVALIDCHK/TREADYCHK/TWAKEUPCHK; TVALID for all data-dependent check signals). The Receiver-side TREADYCHK is monitored by the Master VIP's monitor. Verification Intent: Drive all parity check signals correctly across a full regression and verify the DUT receiver accepts the transfers without error. Inject single-bit flips on TDATACHK, TVALIDCHK, and TLASTCHK individually and verify the VIP monitor detects the parity mismatch and reports UVM_ERROR. Verify that parity signals disabled when their Check Enable is FALSE are not evaluated. Monitor TREADYCHK from the DUT and flag any mismatch against the expected parity of TREADY.
----------------	--	--	--

3. Test Cases & Mapping

Index	Scenario	TR Mapping
TC_MST_001	Master VIP asserts TVALID and drives a valid 32-bit TDATA payload. DUT Slave asserts TREADY in the same cycle. Verify TDATA, TSTRB, TKEEP, TLAST, TID, TDEST are all captured by the monitor as stable on the rising ACLK edge where both TVALID=1 and TREADY=1. Monitor must log no stability violation. Scoreboard verifies payload byte-exact match.	REQ_MST_01
TC_MST_002	Master VIP asserts TVALID with a 64-bit payload. DUT Slave holds TREADY LOW for exactly 5 ACLK cycles before asserting. Monitor samples TDATA on cycles 1-4 and verifies no change. Transfer completes on cycle 5. Scoreboard confirms payload and packet position are correct.	REQ_MST_01
TC_MST_003	Master VIP asserts TVALID with a complete 32-byte packet (4 beats, TDATA=64-bit). DUT Slave holds TREADY LOW for 100 ACLK cycles per beat. Monitor captures TDATA, TSTRB, TKEEP, TID, TDEST at each cycle within the 100-cycle stall window and verifies they remain bit-for-bit constant. Total transfer time = 400 cycles. Scoreboard verifies all 32 bytes match the original sequence item.	REQ_MST_01

Verification Plan Report

TC_MST_004	Enable the seq_item violation knob drop_valid_early=1. Master VIP driver deasserts TVALID on cycle 3 of a 10-cycle TREADY stall without completing the handshake. Verify the VIP monitor fires a UVM_ERROR containing the message TVALID_STABILITY_VIOLATION and identifies the beat address and TID. Verify the scoreboard does not count the incomplete transfer as a valid packet beat.	REQ_MST_01
TC_MST_005	DUT stimulus model (via reactive sequence) pre-asserts TREADY HIGH before Master VIP drives TVALID. Master VIP asserts TVALID with a 1-beat packet (TLAST=1). Verify the transfer completes on the same cycle TVALID is asserted (T_transfer = T_TVALID). Monitor logs beat-count=1 and correct TLAST. Scoreboard confirms single-byte payload match.	REQ_MST_02
TC_MST_006	DUT maintains TREADY=1 throughout. Master VIP generates an 8-beat burst (TLAST on beat 8). Verify all 8 beats complete at one-beat-per-cycle throughput (channel utilization=100%). No idle cycles between beats. Scoreboard reconstructs the 8-beat packet and validates the full payload and correct TLAST position.	REQ_MST_02
TC_MST_007	DUT asserts and deasserts TREADY multiple times (3 pulses) while TVALID remains LOW. Master VIP must not initiate any transfer or change any output signals. After the spurious TREADY pulses, Master VIP asserts TVALID and completes a normal transfer. Verify no phantom transfers are recorded by the monitor or scoreboard during the spurious-TREADY window.	REQ_MST_02
TC_MST_008	Master VIP generates a 64-beat burst. For each beat, the TREADY delay is randomized between 0 and 15 ACLK cycles using constrained randomization. Verify all 64 beats are received correctly by the scoreboard regardless of per-beat delay variation. The monitor must log each beat's stall duration and report overall latency statistics.	REQ_MST_02
TC_MST_009	DUT maintains TREADY=1. Master VIP generates a 256-beat burst at 100% throughput. Monitor measures throughput: exactly 256 transfers in 256 cycles. Scoreboard validates all 256 data beats, TKEEP patterns, and the single TLAST on beat 256. No idle cycles permitted between beat 1 and beat 256.	REQ_MST_03

Verification Plan Report

TC_MST_010	DUT maintains TREADY=1. Master VIP generates 10 consecutive 8-beat packets with no idle cycles between them (TLAST on beat 8 of each packet, TVALID immediately rises for the next packet's beat 1 in the following cycle). Scoreboard reconstructs 10 distinct packets by TLAST boundaries and validates each packet's payload independently.	REQ_MST_03
TC_MST_011	Master VIP generates a single-beat packet with TLAST=1 on the first and only transfer. TDATA contains a fully randomized 32-bit word. TKEEP=0xF (all bytes valid). Scoreboard must detect the packet as exactly 1 beat and validate the payload. Monitor confirms TLAST assertion on beat 1.	REQ_MST_04
TC_MST_012	Master VIP generates a 16-beat packet. TLAST is asserted exclusively on beat 16; beats 1-15 must have TLAST=0. Monitor verifies TLAST=0 on beats 1-15 and TLAST=1 on beat 16. Scoreboard counts the 16 beats and reconstructs the packet payload. Negative check: no early TLAST assertion is permitted.	REQ_MST_04
TC_MST_013	Master VIP generates a 256-beat packet with TLAST on the final beat. TDATA is a constrained-random 64-bit pattern unique per beat. Scoreboard validates all 256 unique data patterns and the single TLAST on beat 256. Verify no buffer overflow condition is triggered in the DUT.	REQ_MST_04
TC_MST_014	Master VIP generates 5 back-to-back 4-beat packets. Immediately after TLAST of packet N, TVALID is asserted for the first beat of packet N+1 with no idle cycle. Scoreboard must identify each of the 5 packets by their TLAST boundaries and validate their independent payloads without merging adjacent packets.	REQ_MST_04
TC_MST_015	Master VIP generates a 7-beat packet where beats 1-6 carry valid data (TKEEP=0xFF, TLAST=0) and beat 7 is a null-termination beat (TLAST=1, all TKEEP=0). Scoreboard must classify the packet as 6 data beats + 1 null-terminator. Verify the DUT acknowledges beat 7 without consuming payload data.	REQ_MST_05
TC_MST_016	Master VIP generates a single-beat packet where TLAST=1 and all TKEEP bits are LOW (pure null packet). Scoreboard must recognize a zero-data-byte packet and not flag a missing payload error. Verify the DUT can merge this null packet with an earlier transfer with the same TID/TDEST and TLAST=0 if merging rules are met.	REQ_MST_05

Verification Plan Report

TC_MST_017	Generate a 4-beat packet where beat 3 has TLAST=0 and TKEEP=0xFF (valid data), and beat 4 has TLAST=1 with all TKEEP=0 (null TLAST). Both beats share same TID and TDEST. Verify the DUT may merge beat 4 into beat 3 per the specification merging rule. Monitor verifies that if merging occurs, no data bytes are lost from beat 3.	REQ_MST_05
TC_MST_018	Master VIP generates a 32-beat burst with TKEEP=0xFF (64-bit bus, all 8 bytes active) on every beat. Verify the scoreboard receives exactly 32*8=256 valid payload bytes. DUT must not suppress any bytes. Monitor confirms TKEEP=0xFF on all beats.	REQ_MST_06
TC_MST_019	Master VIP drives TKEEP=0x55 (alternating bytes active) on all beats of a 16-beat burst. Scoreboard validates that only the 4 active bytes per beat (bytes 0, 2, 4, 6) are counted as valid payload. Total valid payload = 16*4=64 bytes. DUT must strip null bytes 1, 3, 5, 7 per beat.	REQ_MST_06
TC_MST_020	Master VIP drives constrained-random TKEEP on each beat of a 64-beat burst (at least 1 bit HIGH per beat, at least 1 bit LOW per beat). The scoreboard must independently reconstruct the valid payload byte sequence using the per-beat TKEEP mask and verify the DUT output matches exactly. Beat-level TKEEP patterns are logged for debug.	REQ_MST_06
TC_MST_021	Master VIP generates a mid-burst beat where TKEEP=0x00 and TLAST=0. Per specification, this transfer with all TKEEP bits LOW can be suppressed by interconnect. Verify the DUT processes or suppresses the beat correctly. Scoreboard accounts for the null beat as contributing zero payload bytes. No UVM_ERROR expected for a legal null transfer.	REQ_MST_06
TC_MST_022	Master VIP generates a 16-beat packet with all bytes as data bytes (TKEEP=0xFF, TSTRB=0xFF on every beat). Scoreboard verifies all 128 bytes are valid data bytes. Monitor confirms the TKEEP=1/TSTRB=1 combination on every beat. DUT must receive all bytes as valid payload.	REQ_MST_07
TC_MST_023	Master VIP generates transfers with bytes 0 and 4 as position bytes (TKEEP[0]=1, TSTRB[0]=0; TKEEP[4]=1, TSTRB[4]=0) and the remaining bytes as data bytes on an 8-beat burst. Scoreboard maps data bytes vs position bytes per the TSTRB mask and verifies the DUT correctly interprets positional alignment. Position byte TDATA content is marked as non-comparable in the scoreboard.	REQ_MST_07

Verification Plan Report

TC_MST_024	Enable the violation knob inject_invalid_tstrb_tkeep=1. Master VIP driver sets TSTRB[2]=1 with TKEEP[2]=0 on a specific beat. Verify the VIP monitor immediately fires a UVM_ERROR with message ILLEGAL_TSTRB_TKEEP_COMBINATION and identifies the beat number and byte position. Scoreboard does not count this beat as a valid transfer.	REQ_MST_07
TC_MST_025	Master VIP generates a 4-beat packet where byte positions alternate between data bytes (TKEEP=1, TSTRB=1) and position bytes (TKEEP=1, TSTRB=0) per a constrained-random pattern. Scoreboard reconstructs the partial-update data structure using the TSTRB mask and validates that only the data bytes are passed to the reference model for payload comparison.	REQ_MST_07
TC_MST_026	Master VIP generates a 32-beat packet with TID=0x3 and TDEST=0x1 on every beat. Monitor verifies TID and TDEST are constant across all 32 beats (intra-packet stability). Scoreboard associates all 32 beats with stream (TID=3, TDEST=1). Transfer completes with correct TLAST on beat 32.	REQ_MST_08
TC_MST_027	Master VIP interleaves two logical streams: Stream A (TID=0x1, 8 beats) and Stream B (TID=0x2, 8 beats) alternating on a per-transfer basis. Beat sequence: A, B, A, B, ... for 16 total beats. Each stream has its own TLAST. Scoreboard demultiplexes by TID and independently validates Stream A payload (8 beats) and Stream B payload (8 beats). Monitor confirms TID changes between beats do not trigger a false violation (inter-packet interleaving is permitted).	REQ_MST_08
TC_MST_028	Master VIP generates 4 simultaneous streams (TID=0, 1, 2, 3) interleaved in round-robin order: 0, 1, 2, 3, 0, 1, 2, 3,... for 32 total beats. Each stream contains 8 beats with a unique TLAST. Scoreboard independently reconstructs each of the 4 streams. Monitor verifies that TID changes between consecutive beats do not produce false protocol violations.	REQ_MST_08
TC_MST_029	Enable violation knob change_tid_mid_packet=1. Master VIP drives a 4-beat packet where beat 1 has TID=0x1 and beat 2 (TLAST=0) has TID=0x2. Verify the VIP monitor fires a UVM_ERROR with message TID_CHANGED_MID_PACKET. Scoreboard does not validate the corrupted packet.	REQ_MST_08
TC_MST_030	Enable Continuous_Packets=True in agent_config. Master VIP generates a 16-beat packet with fixed TID=0x5 and TDEST=0x2 across all beats (TLAST=0 for beats 1-15, TLAST=1 on beat 16). Monitor verifies TID and TDEST do not change while TLAST=0. Scoreboard validates the full packet payload.	REQ_MST_09

Verification Plan Report

TC_MST_031	Enable Continuous_Packets=True. Master VIP generates an 8-beat packet. For beats 1-7 (TLAST=0), TKEEP must be all-HIGH (0xFF). On beat 8 (TLAST=1), TKEEP may have null bytes only in the high-order byte positions. Monitor flags any beat with TLAST=0 and TKEEP not all-HIGH as a UVM_ERROR. Scoreboard validates the packet payload.	REQ_MST_09
TC_MST_032	Enable Continuous_Packets=True and inject TID_change violation. Master VIP drives a 4-beat burst where TID=0x1 on beats 1-2 and TID=0x3 on beats 3-4 with TLAST=0 on all beats except beat 4. Monitor must detect the TID change when TLAST=0 and fire a UVM_ERROR with CONTINUOUS_PKT_TID_CHANGE_VIOLATION.	REQ_MST_09
TC_MST_033	Enable Continuous_Packets=True. Master VIP generates a 5-beat packet where beat 5 (TLAST=1) has TKEEP=0x1F (lower 5 bytes valid, upper 3 bytes null). Verify monitor accepts this as a legal continuous-packet final beat. Scoreboard reconstructs the partial final beat with the correct byte count.	REQ_MST_09
TC_MST_034	Assert ARESETn LOW while Master VIP is idle (TVALID=0). Verify TVALID remains 0 throughout the reset window (10 ACLK cycles). Deassert ARESETn HIGH. Verify Master VIP does not drive TVALID HIGH until the ACLK cycle after ARESETn deassertion. Monitor confirms correct post-reset TVALID timing.	REQ_MST_10
TC_MST_035	Master VIP begins a 16-beat transfer (4 beats completed). ARESETn is asserted LOW on beat 5. Verify TVALID is driven LOW immediately (by the next ACLK cycle). Scoreboard flushes the in-progress 4 completed beats and does not raise a payload mismatch error for the cancelled beats. Monitor logs the abort event.	REQ_MST_10
TC_MST_036	Assert ARESETn LOW for 5 cycles then deassert HIGH at T0. Verify the Master VIP does not drive TVALID HIGH at T0 (same cycle as ARESETn deassertion). TVALID must first go HIGH at T1 (the next ACLK rising edge after T0) or later. Verify that a complete transfer is successfully executed starting from T1.	REQ_MST_10
TC_MST_037	Execute 5 consecutive reset cycles (ARESETn LOW for 3 cycles then HIGH for 5 cycles each) while the Master VIP alternately generates 4-beat bursts between resets. Verify TVALID correctly deasserts during each reset, and recovers correctly after each deassertion. All complete packets between resets must be validated by the scoreboard.	REQ_MST_10

Verification Plan Report

TC_MST_038	Master VIP asserts TWAKEUP at T0. TVALID is asserted at T1 (one cycle after T0). Verify TWAKEUP is HIGH before TVALID. DUT de-gates clock and responds with TREADY at T3. Monitor verifies TWAKEUP remained continuously asserted from T0 through T3 (until TREADY handshake). Verify no glitches on TWAKEUP during the window.	REQ_MST_11
TC_MST_039	Master VIP asserts both TWAKEUP and TVALID in the same ACLK cycle. DUT holds TREADY LOW for 8 cycles. Verify TWAKEUP remains asserted for all 8 cycles until TREADY is asserted. Monitor fires a UVM_ERROR if TWAKEUP drops before TREADY is sampled HIGH. Transfer completes correctly.	REQ_MST_11
TC_MST_040	Master VIP completes a 4-packet burst. After the final TLAST handshake, TVALID goes LOW and TWAKEUP is deasserted 2 cycles later. Monitor verifies TWAKEUP is only deasserted after TVALID goes LOW (no further transfers in progress). Verify the DUT can re-gate clock after TWAKEUP deassertion.	REQ_MST_11
TC_MST_041	Master VIP asserts TWAKEUP for 10 cycles without ever asserting TVALID, then deasserts TWAKEUP. Verify no deadlock occurs (no TVALID means DUT need not assert TREADY). Verify the monitor does not raise a spurious error for this permitted-but-not-recommended usage. Verify system recovers and subsequent normal transfers complete correctly.	REQ_MST_11
TC_MST_042	Master VIP drives TDATA=0x00000000...00 (64-bit, all zeros). For odd parity, each byte parity bit must be HIGH (odd parity of 0x00 = 1 parity bit HIGH to make count odd). Verify TDATACHK = 0xFF (all parity bits HIGH). Monitor confirms correct parity on every cycle TVALID is HIGH. DUT accepts the transfer without error.	REQ_MST_12
TC_MST_043	Master VIP drives TDATA=0xFFFFFFFFFFFFFFFF. For odd parity, each byte (0xFF = 8 HIGH bits, even number) gets a parity bit of HIGH to make the total count odd. TDATACHK=0xFF. Monitor verifies. DUT accepts. Verify TVALIDCHK=1 (TVALID=1, odd count=1, parity=0 to make it odd ? TVALIDCHK=0).	REQ_MST_12
TC_MST_044	Master VIP generates 100 constrained-random TDATA values across a 100-beat burst. For each beat, the driver computes TDATACHK using the odd-parity formula per byte and drives it. Monitor independently computes expected TDATACHK from observed TDATA and compares. Zero mismatches expected. Report a UVM_ERROR for any mismatch.	REQ_MST_12

Verification Plan Report

TC_MST_045	Enable parity_inject_error=1 on a specific beat (beat 8). Driver deliberately flips TDATACHK[3] (inverting the parity of byte 3). Monitor observes the TDATA-to-TDATACHK relationship and fires a UVM_ERROR with TDATACHK_PARITY_ERROR identifying beat 8 and byte 3. Verify the DUT error-handling port (if present) is also asserted.	REQ_MST_12
TC_MST_046	During a 64-beat burst, monitor captures TVALIDCHK on every cycle (ARESETn=HIGH) and verifies it is the correct odd parity of TVALID. Similarly, monitor verifies TLASTCHK on every cycle where TVALID=HIGH. Scoreboard reports any cycle where TVALIDCHK or TLASTCHK does not match the expected odd parity as a UVM_ERROR.	REQ_MST_12
TC_MST_047	During normal transfers, Master VIP monitor observes TREADYCHK driven by the DUT. Monitor computes expected TREADYCHK from observed TREADY (odd parity of 1-bit signal). Verify TREADYCHK matches expected parity on every cycle where ARESETn=HIGH. Flag any mismatch as TREADYCHK_PARITY_ERROR. This validates the DUT Slave Receiver's parity generation logic.	REQ_MST_12
TC_MST_048	Master VIP generates a 32-beat packet with all AXI5 parity signals driven correctly: TVALIDCHK, TDATACHK (64-bit bus, 8 parity bits), TSTRBCHK, TKEEPCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TWAKEUPCHK. Monitor validates all parity signals on every beat. Scoreboard confirms the DUT Slave receives the full packet without error. Zero parity violations expected. This serves as the golden-path end-to-end parity regression.	REQ_MST_12

4. Functional Coverage Groups

Group Name	Description
cg_tvalid_stability	Measures correctness of TVALID stability after assertion. Covers zero-stall, moderate-stall (1-20 cycles), and maximum-stall (>20 cycles) TREADY delay categories. Cross-coverage between stall duration and data bus width.
cg_handshake_scenarios	Captures all three canonical handshake sequences from the AXI-Stream specification. TVALID-before-TREADY (Fig 2-1), TREADY-before-TVALID (Fig 2-2), and simultaneous TVALID+TREADY (Fig 2-3). Cross-coverage with packet length (1, 2-8, 9-64, >64 beats).
cg_packet_boundary	Measures TLAST assertion patterns including 1-beat, short (2-8 beats), medium (9-64 beats), and maximum (65-256 beats) packet lengths. Includes back-to-back packet coverage bins and null-termination (TLAST=1, all-TKEEP=0) bins.

Verification Plan Report

cg_tkeep_patterns	Covers TKEEP generation across four categories -- all-HIGH (dense), all-LOW (null), alternating (50% utilization), and sparse random. Cross-coverage between TKEEP pattern and TLAST state (mid-packet vs boundary beat).
cg_tstrb_qualification	Covers all four TKEEP/TSTRB byte combinations from Table 2-3 -- data byte (1,1), position byte (1,0), null byte (0,0), and illegal reserved (0,1, negative only). Cross-coverage between byte position index and qualification type per transfer.
cg_stream_interleaving	Covers TID stream multiplexing scenarios -- single stream, 2-stream interleave, 4-stream interleave, 8-stream interleave. Cross-coverage between interleave depth and packet length. Includes intra-packet TID stability and inter-packet TID change bins.
cg_continuous_packets	Covers continuous packet mode compliance -- enabled vs disabled, TKEEP all-HIGH in non-final beats, null bytes only on final beat, no TID/TDEST change mid-packet. Includes violation bins for detected errors.
cg_reset_scenarios	Covers all reset scenarios -- idle-state reset, mid-transfer reset, mid-packet reset. Covers post-reset TVALID timing (correct delay of at least 1 cycle after ARESETn deassertion). Covers multiple consecutive reset cycles.
cg_wakeup_timing	Covers TWAKEUP temporal relationships -- pre-TVALID assertion, simultaneous with TVALID, assertion without transfer. Covers TWAKEUP duration until TREADY handshake completes. Includes glitch-detection coverage (no mid-cycle transitions observed).
cg_parity_signals	Covers parity signal correctness for each AXI5 check signal -- TVALIDCHK, TDATACHK (per-byte coverage), TLASTCHK, TSTRBCHK, TKEEPCHK, TIDCHK, TDESTCHK, TUSERCHK, TWAKEUPCHK. Includes error-injection bins for each signal type and TREADYCHK monitoring.
cg_data_bus_width	Cross-coverage group ensuring all features are exercised across all configured data bus widths (8, 16, 32, 64, 128, 256 bits). Crossed with packet length and TKEEP density to ensure full combinatorial coverage of bus utilization scenarios.
cg_negative_violations	Dedicated violation coverage group. Each bin represents a distinct protocol violation category -- TVALID_DROP, ILLEGAL_TSTRB_TKEEP, TID_MID_PACKET_CHANGE, TDATACHK_ERROR, TVALIDCHK_ERROR, CONTINUOUS_PKT_TID_CHANGE, TWAKEUP_DROPPED_BEFORE_TREADY. Ensures all negative scenarios are exercised at least once per regression.

5. Interface Signals

Signal Name	Direction	Width	Description
ACLK	Input	1	Global clock signal. All VIP output signals change after the rising edge of ACLK. All input signals (TREADY, TREADYCHK) are sampled on the rising edge of ACLK.

Verification Plan Report

ARESETn	Input	1	Active-LOW synchronous reset. Deassertion must be synchronous after the rising edge of ACLK. During reset, the Master VIP must drive TVALID LOW.
TVALID	Output	1	Master VIP drives HIGH to indicate a valid transfer is present. Once asserted, TVALID must remain HIGH until the handshake completes (TREADY sampled HIGH on rising ACLK edge). The Master is not permitted to wait for TREADY before asserting TVALID.
TREADY	Input	1	DUT Slave Receiver drives HIGH to signal it can accept a transfer. The Master VIP observes this signal to detect back-pressure. Default value is HIGH when the DUT omits TREADY.
TDATA	Output	TDATA_WIDTH	Primary payload bus driven by the Master VIP Transmitter. TDATA_WIDTH must be an integer number of bytes (8, 16, 32, 64, 128, 256, 512, or 1024 bits). All data bytes must remain stable from the assertion of TVALID until the transfer completes.
TSTRB	Output	TDATA_WIDTH /8	Byte qualifier driven by Master VIP. TSTRB[x] HIGH indicates TDATA[(8x+7):8x] is a data byte. TSTRB[x] LOW (with TKEEP[x] HIGH) indicates a position byte. The combination TSTRB=HIGH with TKEEP=LOW is reserved and must not be generated.
TKEEP	Output	TDATA_WIDTH /8	Byte qualifier driven by Master VIP. TKEEP[x] HIGH indicates the associated byte must be transported to the destination. TKEEP[x] LOW indicates a null byte that can be removed from the stream. A transfer with all TKEEP bits LOW (null transfer) is permitted.
TLAST	Output	1	Packet boundary signal driven by Master VIP. When asserted, indicates the final beat of a packet. The number of TLAST assertions must be preserved between Transmitter and Receiver. The start of the next packet begins implicitly on the cycle following TLAST assertion.
TID	Output	TID_WIDTH	Data stream identifier driven by Master VIP. Recommended maximum TID_WIDTH is 8 bits. All bytes within a packet must have the same TID value. Interleaving of streams with different TID values is permitted on a per-transfer basis and is not limited to TLAST boundaries.
TDEST	Output	TDEST_WIDTH	Routing destination signal driven by Master VIP. Recommended maximum TDEST_WIDTH is 8 bits. Transfers within the same packet must have consistent TDEST values. Merging of transfers with different TID or TDEST values is not permitted.

Verification Plan Report

TUSER	Output	TUSER_WIDTH	User-defined sideband information driven by Master VIP along with the data stream. Recommended TUSER_WIDTH is an integer multiple of TDATA_WIDTH/8. TUSER information must remain associated with the same byte position across upsizing or downsizing operations.
TWAKEUP	Output	1	Activity indicator signal driven by Master VIP for AXI5-Stream interfaces. If both TWAKEUP and TVALID are HIGH in the same cycle, TWAKEUP must remain asserted until TREADY is asserted. A Receiver is permitted to wait for TWAKEUP before asserting TREADY, creating a potential deadlock risk if TWAKEUP is never asserted.
TVALIDCHK	Output	1	Odd parity check signal for TVALID, driven by Master VIP. Must be correctly driven in every cycle where ARESETn is HIGH. Applies to AXI5-Stream interfaces with Check_Type set to Odd_Parity_Byte_All.
TDATACHK	Output	TDATA_WIDTH /8	Odd parity check signal for TDATA, driven by Master VIP. Each bit of TDATACHK covers 8 bits of TDATA. Must be driven correctly in every cycle where TVALID is HIGH.
TSTRBCHK	Output	ceil(TDATA_WIDTH/64)	Odd parity check signal for TSTRB, driven by Master VIP. Granularity covers 1-8 TSTRB bits per check bit. Must be driven correctly whenever TVALID is HIGH.
TKEEPCHK	Output	ceil(TDATA_WIDTH/64)	Odd parity check signal for TKEEP, driven by Master VIP. Must be driven correctly whenever TVALID is HIGH.
TLASTCHK	Output	1	Odd parity check signal for TLAST, driven by Master VIP. 1-bit granularity. Must be driven correctly whenever TVALID is HIGH.
TIDCHK	Output	ceil(TID_WIDTH/8)	Odd parity check signal for TID, driven by Master VIP. Must be driven correctly whenever TVALID is HIGH.
TDESTCHK	Output	ceil(TDEST_WIDTH/8)	Odd parity check signal for TDEST, driven by Master VIP. Must be driven correctly whenever TVALID is HIGH.
TUSERCHK	Output	ceil(TUSER_WIDTH/8)	Odd parity check signal for TUSER, driven by Master VIP. Must be driven correctly whenever TVALID is HIGH.
TWAKEUPCHK	Output	1	Odd parity check signal for TWAKEUP, driven by Master VIP. Must be driven correctly in every cycle where ARESETn is HIGH.
TREADYCHK	Input	1	Odd parity check signal for TREADY, driven by DUT Slave Receiver. The Master VIP monitor observes TREADYCHK to detect parity errors on the TREADY signal path. Must be driven correctly in every cycle where ARESETn is HIGH.

Verification Plan Report

6. Verification Environment Structure

```
axi_stream_master_vip_tb/
|-- master_agent/
|   |-- axi_stream_master_vip_agent.sv
|   |-- axi_stream_master_vip_agent_config.sv
|   |-- axi_stream_master_vip_driver.sv
|   |-- axi_stream_master_vip_monitor.sv
|   |-- axi_stream_master_vip_sequencer.sv
|   |-- axi_stream_master_vip_callback.sv
|-- env/
|   |-- axi_stream_master_vip_scoreboard.sv
|   |-- axi_stream_master_vip_env_config.sv
|   |-- axi_stream_master_vip_env.sv
|-- sequences/
|   |-- axi_stream_master_vip_seq_item.sv
|   |-- axi_stream_master_vip_base_sequence.sv
|   |-- axi_stream_master_vip_test_sequences.sv
|-- top/
|   |-- axi_stream_master_vip_defines.sv
|   |-- axi_stream_master_vip_pkg.sv
|   |-- axi_stream_master_vip_if.sv
|   |-- axi_stream_master_vip_tb_top.sv
|   |-- axi_stream_master_vip_test.sv
```

7. SystemVerilog Implementation

axi_stream_master_vip_defines.sv (Architecture Blueprint)

COMPONENT: Global Preprocessor Defines

MEMBERS:

```
- `define AXI_DATA_WIDTH      64
- `define AXI_STRB_WIDTH      (AXI_DATA_WIDTH/8)
- `define AXI_TID_WIDTH      8
- `define AXI_TDEST_WIDTH     4
- `define AXI_TUSER_WIDTH     8
- `define HAS_TKEEP           1
- `define HAS_TSTRB           1
- `define HAS_TLAST           1
- `define HAS_TID             1
- `define HAS_TDEST           1
- `define HAS_TUSER           1
- `define HAS_TWAKEUP         1
- `define HAS_PARITY          1
- `define AXI_MAX_PACKET_BEATS 256
- `define UVM_TIMEOUT_NS      100000000
```

NOTES:

- All architectural widths sourced from these defines for global visibility.
- HAS_* flags drive conditional compilation in driver and monitor.

Verification Plan Report

axi_stream_master_vip_seq_item.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_seq_item

EXTENDS: uvm_sequence_item

MEMBERS:

- [rand logic [AXI_DATA_WIDTH-1:0]] tdata
- [rand logic [AXI_STRB_WIDTH-1:0]] tstrb
- [rand logic [AXI_STRB_WIDTH-1:0]] tkeep
- [rand logic] tlast
- [rand logic [AXI_TID_WIDTH-1:0]] tid
- [rand logic [AXI_TDEST_WIDTH-1:0]] tdest
- [rand logic [AXI_TUSER_WIDTH-1:0]] tuser
- [rand int unsigned] tready_delay (0..15 cycles)
- [rand int unsigned] packet_length (1..256 beats)
- [bit] drop_valid_early (violation knob)
- [bit] inject_invalid_tstrb_tkeep (violation knob)
- [bit] change_tid_mid_packet (violation knob)
- [bit] parity_inject_error (violation knob)
- [int] parity_error_byte_idx

CONSTRAINTS:

- tkeep != 0 unless drop_valid_early=1 (structural constraint)
- tstrb == 0 when tkeep == 0 (null byte law)
- tstrb[x] == 0 when tkeep[x] == 0 (per-bit position law, unless inject_invalid=1)
- packet_length inside {1, [2:8], [9:64], [65:256]}
- tready_delay inside {0, [1:5], [6:20], [21:100]}

METHODS:

- do_copy: deep copy all rand fields and all violation knobs
- do_compare: compare tdata, tstrb, tkeep, tlast, tid, tdest, tuser only
- convert2string: multi-line formatted dump of all fields

axi_stream_master_vip_agent_config.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_agent_config

EXTENDS: uvm_object

MEMBERS:

- [virtual axi_stream_master_vip_if] vif
- [uvm_active_passive_enum] is_active = UVM_ACTIVE
- [bit] enable_parity = 1
- [bit] continuous_packets_mode = 0
- [bit] has_twakeup = 1
- [int] max_packet_beats = 256
- [int] tready_max_stall = 100
- [bit] enable_protocol_checks = 1

NOTES:

- Fetched via uvm_config_db in build_phase of agent.
- All HAS_* macros map to corresponding config knobs for dynamic control.

axi_stream_master_vip_driver.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_driver

Verification Plan Report

```
EXTENDS: uvm_driver #(axi_stream_master_vip_seq_item)
CONFIG_REF: axi_stream_master_vip_agent_config (Source: ConfigDB)
MEMBERS:
- [virtual axi_stream_master_vip_if] vif
- [axi_stream_master_vip_agent_config] cfg
- [int] packet_count
METHODS:
- build_phase: get cfg from ConfigDB, get vif from cfg
- run_phase/body:
  LOOP forever:
    seq_item_port.get_next_item(req)
    CALL drive_packet(req)
    seq_item_port.item_done()
- drive_packet(item):
  FOR beat = 1 to item.packet_length:
    @(posedge vif.ACLK);
    vif.TVALID <= 1
    vif.TDATA <= item.tdata (beat-specific randomization)
    vif.TSTRB <= item.tstrb
    vif.TKEEP <= item.tkeep
    vif.TLAST <= (beat == item.packet_length) ? 1 : 0
    vif.TID <= item.tid
    vif.TDEST <= item.tdest
    vif.TUSER <= item.tuser
    IF cfg.has_twakeup: drive_twakeup()
    IF cfg.enable_parity: compute_and_drive_parity()
    IF item.drop_valid_early AND beat==3: vif.TVALID <= 0 (violation path)
    IF item.inject_invalid_tstrb_tkeep: force illegal tstrb/tkeep combo
    WAIT for TREADY (monitor stall, up to cfg.tready_max_stall cycles)
    vif.TVALID <= 0
- compute_and_drive_parity():
  TDATACHK[i] = ^vif.TDATA[8i+7:8i] ^ 1 (odd parity per byte)
  TVALIDCHK = ^vif.TVALID ^ 1
  TLASTCHK = ^vif.TLAST ^ 1
  TIDCHK, TDESTCHK, TUSERCHK, TWAKEUPCHK computed similarly
```

axi_stream_master_vip_monitor.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_vip_monitor
EXTENDS: uvm_monitor
CONFIG_REF: axi_stream_master_vip_agent_config (Source: ConfigDB)
MEMBERS:
- [virtual axi_stream_master_vip_if] vif
- [uvm_analysis_port #(axi_stream_master_vip_seq_item)] ap
- [int] beat_counter, packet_counter
- [logic [AXI_DATA_WIDTH-1:0]] prev_tdata
METHODS:
- run_phase:
  LOOP forever:
    @(posedge vif.ACLK);
```

Verification Plan Report

```
IF ARESETn==0: reset_state(); continue
IF TVALID==1:
    CHECK stability: IF TVALID was 1 last cycle AND handshake NOT complete:
        verify TDATA/TSTRB/TKEEP/TID/TDEST unchanged vs prev_tdata ? else UVM_ERROR
    IF cfg.enable_parity: verify_parity_signals()
    IF TVALID==1 AND TREADY==1:
        capture beat into seq_item
        beat_counter++
        IF TLAST==1: broadcast packet to ap; packet_counter++; beat_counter=0
    save prev_tdata, prev_tvalid
- verify_parity_signals():
    FOR each byte i: expected_datachk[i] = ^TDATA[8i+7:8i] ^ 1
    IF TDATACHK != expected_datachk: UVM_ERROR("TDATACHK_PARITY_ERROR")
    Verify TVALIDCHK, TLASTCHK, TIDCHK, TDESTCHK similarly
    Observe TREADYCHK from DUT: verify against ^TREADY^1
- reset_state(): flush beat_counter, log MONITOR_RESET_FLUSH
- write_packet_tracker_file(): log each completed packet to tracker CSV file
```

axi_stream_master_vip_scoreboard.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_vip_scoreboard
EXTENDS: uvm_scoreboard
MEMBERS:
- [uvm_analysis_imp] master_ap_imp (receives from monitor)
- [axi_stream_master_vip_seq_item queue [$]] expected_queue [tid] (associative by TID)
- [int] pass_count, fail_count, null_packet_count
METHODS:
- write(item):
    Reconstruct packet payload from captured beats, excluding null bytes (TKEEP=0).
    Exclude position bytes (TSTRB=0 with TKEEP=1) from data comparison.
    Pop matching item from expected_queue[item.tid]
    Compare TDATA payload (byte-accurate), TID, TDEST, packet length
    IF mismatch: UVM_ERROR with detailed field-level diff
    IF item is null-terminator (all TKEEP=0, TLAST=1): log as null_packet, no payload compare
    Pass: pass_count++; Fail: fail_count++
- check_phase: IF fail_count > 0: UVM_ERROR("REGRESSION FAILED: N failures")
- flush_on_reset(): clear expected_queue, log SCOREBOARD_FLUSHED
```

axi_stream_master_vip_base_sequence.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_vip_base_sequence
EXTENDS: uvm_sequence #(axi_stream_master_vip_seq_item)
MEMBERS:
- [int] num_packets = 10
- [int] beats_per_packet = 8
- [bit] enable_interleaving = 0
- [bit] continuous_mode = 0
METHODS:
- body():
```

Verification Plan Report

```
REPEAT num_packets:
    req = axi_stream_master_vip_seq_item::type_id::create("req")
    start_item(req)
    assert(req.randomize() with { packet_length == beats_per_packet; })
    IF continuous_mode: req.tid = fixed_tid; req.tdest = fixed_tdest
    finish_item(req)
```

axi_stream_master_vip_test_sequences.sv (Architecture Blueprint)

COMPONENT: One sequence class per test case TC_MST_001 through TC_MST_048

NAMING: axi_stream_master_vip_tc_mst_<NNN>_seq extends axi_stream_master_vip_base_sequence

EXAMPLES:

```
axi_stream_master_vip_tc_mst_001_seq:
    body(): drive 1 single-beat packet, TREADY stall=0
axi_stream_master_vip_tc_mst_004_seq:
    body(): enable drop_valid_early knob; drive 1 packet; expect monitor UVM_ERROR
axi_stream_master_vip_tc_mst_027_seq:
    body(): generate 4-stream interleaved traffic, 8 beats per stream
axi_stream_master_vip_tc_mst_045_seq:
    body(): enable parity_inject_error on beat 8; expect TDATACHK_PARITY_ERROR
```

axi_stream_master_vip_test.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_base_test extends uvm_test

COMPONENT: One test class per TC: axi_stream_master_vip_tc_mst_<NNN>_test

NAMING: axi_stream_master_vip_tc_mst_001_test, ..., axi_stream_master_vip_tc_mst_048_test

METHODS:

- build_phase: create env; set agent_config via ConfigDB
- run_phase: create and start the matching tc_mst_NNN_seq on the master sequencer
- Each test maps 1:1 to its corresponding tc_mst_NNN_seq

axi_stream_master_vip_if.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_if (SystemVerilog Interface)

PORTS:

- input logic ACLK, ARESETn
- logic TVALID, TREADY
- logic [AXI_DATA_WIDTH-1:0] TDATA
- logic [AXI_STRB_WIDTH-1:0] TSTRB, TKEEP
- logic TLAST
- logic [AXI_TID_WIDTH-1:0] TID
- logic [AXI_TDEST_WIDTH-1:0] TDEST
- logic [AXI_TUSER_WIDTH-1:0] TUSER
- logic TWAKEUP
- logic TVALIDCHK, TREADYCHK
- logic [AXI_STRB_WIDTH-1:0] TDATACHK
- logic TLASTCHK, TWAKEUPCHK

CLOCKING BLOCK: No explicit skew directives; rely on posedge ACLK edge sampling.

Verification Plan Report

axi_stream_master_vip_pkg.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_pkg (SystemVerilog Package)

INCLUDES (in dependency order):

- `include "axi_stream_master_vip_defines.sv"
- import uvm_pkg::*
- `include "axi_stream_master_vip_seq_item.sv"
- `include "axi_stream_master_vip_agent_config.sv"
- `include "axi_stream_master_vip_sequencer.sv"
- `include "axi_stream_master_vip_driver.sv"
- `include "axi_stream_master_vip_callback.sv"
- `include "axi_stream_master_vip_monitor.sv"
- `include "axi_stream_master_vip_agent.sv"
- `include "axi_stream_master_vip_env_config.sv"
- `include "axi_stream_master_vip_scoreboard.sv"
- `include "axi_stream_master_vip_env.sv"
- `include "axi_stream_master_vip_base_sequence.sv"
- `include "axi_stream_master_vip_test_sequences.sv"
- `include "axi_stream_master_vip_test.sv"

axi_stream_master_vip_tb_top.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_vip_tb_top (Top-Level Simulation Module)

MEMBERS:

- reg ACLK, ARESETn (100MHz clock generator)
- axi_stream_master_vip_if dut_if(.ACLK(ACLK), .ARESETn(ARESETn))
- DUT stub: axi_stream_slave_dut u_dut(.TVALID, .TREADY, .TDATA, ...)

METHODS:

- initial: uvm_config_db set vif; run_test()
- ARESETn: assert LOW for 10 cycles, then HIGH synchronously